Open Access | https://doi.org/10.25080/eyaj7367



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

On-boarding and retaining maintainer talent for MNE-Python

Britta U. Westner[1,2], **Eric Larson**[3], and **Daniel McCloy**[3]

[1]Donders Institute for Brain, Cognition and Behavior, [2]Radboud University Medical Center, [3]Institute for Learning and Brain Sciences, University of Washington

Abstract

MNE-Python is open-source software for analyzing electrophysiological data in neuroscience (the name refers to Minimum Norm Estimation, though the package does much more than that). Like many projects, we struggle to attract and retain maintainers. Besides the usual reasons (overworked researchers facing career instability and a publication-focused incentive structure), our challenge is especially acute: most neuroscientists lack software maintenance expertise, and most software engineers lack neuroscience domain knowledge. To address this, we organized progressive training sprints over several years. Currently, four alumni of those sprints are being onboarded as maintainers. We are seeing positive outcomes from this approach, but at a high cost. We are now developing a curriculum to streamline future onboarding efforts.

Keywords neuroscience, software maintenance, software sustainability, open source education, community management

1. Background

MNE-Python [1] is open-source software for analyzing electrophysiological data in human neuroscience. “MNE” is an initialism for “minimum norm estimation”, one of several ways in which activity in the brain can be estimated from measurements of the electromagnetic field taken outside the head. The name is historical; nowadays MNE-Python implements many such inverse imaging algorithms, alongside analysis and visualization functionality described below.

MNE-Python was created in 2010 by Alexandre Gramfort as a port of the original MNE software (written in C by Matti Hämäläinen). The package is pure Python with no compiled code, and wraps many foundational Scientific Python libraries (NumPy [2], SciPy [3], Scikit-Learn [4], [5], Pandas [6], [7], Statsmodels [8], and others) with 2D visualizations via Matplotlib [9] and PyQtGraph [10], and 3D visualizations via PyVista [11]. The 1.12.1 release (2026-04-20) contained 130,089 lines of Python source code, 72,042 lines of test code, 68,597 lines of docstrings and comments, and 318 pages of documentation (of which 204 are example/tutorial notebooks), as computed by `cloc` [12].

MNE-Python’s initial emphasis was magneto- and electro-encephalography (MEG and EEG) signals, but has since expanded to encompass electrocorticography (ECoG), functional near-infrared spectroscopy (fNIRS), and eyetracking data as well. It provides functionality for pre-processing (including filtering, downsampling, artifact detection and suppression), signal analysis (time-domain, spectral, spectrotemporal, clustering, decoding, and more), inverse imaging (estimation of cortical sources based on external sensor signals), and visualization. MNE-Python supports reading structural magnetic resonance (MR) images for visualizing estimated neural activity in the context of individual subjects’ anatomy, but does *not* perform analysis of MR images, MR spectroscopic images, diffusion tensor images

Published Jul 13, 2026

Correspondence to
Daniel McCloy
dan@mccloy.info

Open Access

Copyright © 2026 Westner *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

(DTI), or other MR-based neuroimaging methods; these modalities are well-covered by other Python neuroimaging software (*e.g.*, `nibabel`, `nilearn`, `nipy`, `nipype`, `DIPY`, and many others).

1.1. Ecosystem

MNE-Python's functionality is further expanded by more than 50 compatible satellite packages. Examples include extensions for specialized analyses like brain connectivity [13], microstate analysis [14], [15], representational similarity analysis [16], or phase-amplitude coupling [17]; for specific data types like intracranial electrodes [18] or near-infrared spectroscopy [19]; for specific data resources like OpenNeuro [20] or the Human Connectome Project [21]; for organizing data to conform to the Brain Imaging Data Structure standard [22], [23]; for handling real-time data streams [24]; and for managing data processing pipelines for large datasets [25], [26], [27]. Many of these related packages are hosted within the MNE-Tools organization on GitHub, and looked after by members of the MNE-Python Maintainer Team.

1.2. Community and Governance

In 2024, the MNE-Python governance changed from a BDFL-plus-maintainers model[1] to a more decentralized power-sharing model comprising an advisory board, a steering council, and a maintainer team. Our user base spans neuroscience research, clinical neurology, and applied neurotechnology, and is estimated to be in the range of 5000-10000 users. The community convenes mostly online through a Q&A forum, biweekly live online office hours, our repositories' issue trackers, and occasional in-person small-group sprints. MNE-Python has no community manager; to the extent that community management happens at all, it is handled mostly by the steering council chair.

2. Problem Statement

Like many open-source software projects, MNE-Python struggles to retain maintainers and reach a comfortable Truck Factor [28]. Part of the problem is that the pool is small: contributors and maintainers need relevant neuroscience knowledge (in order to implement sensible default behavior, stay on top of new methodological developments, and accurately communicate trade-offs for different analysis choices) and also need software development expertise (not just "writing good code", but also knowledge of testing, continuous integration and deployment, dependency and security management, and related topics). Since most MNE-Python users are not formally trained in software engineering, finding suitable candidates is difficult. The problem of attracting and retaining contributors is further aggravated by academic incentive systems which devalue open-source work compared to scientific publications [29]; those barriers to open-source contributions in academia also reinforce existing representational biases in developer demographics, since some groups (such as women, Non-White persons, more senior academics) face greater obstacles to contributing [29], [30].

In the past, regular contributors often came from research groups that already had strong ties to the MNE-Python development team, or groups where the principal investigator had a vested interest in contributing. While those contributors usually had a better-than-average set of relevant skills, this network-based approach almost surely missed many promising contributors. This approach is also reliant on social ties that may atrophy when key network members experience career or life changes, making the approach somewhat brittle. Accordingly, we sought to mitigate the risk posed by low maintainer numbers by striving to remove some of the barriers to MNE-Python contribution — especially those related to (lack of) open-source contribution skills. Therefore, our task became finding interested

[1] The founding BDFL was Alexandre Gramfort, replaced by Daniel McCloy in 2022.

neuroscientists and teaching them how to develop and maintain scientific software.[2] Our efforts to achieve this are the topic of the rest of this paper; in brief, we sought funding for and carried out a multi-year series of progressive training events culminating in the onboarding of four new maintainers.

3. Interventions

To increase our contributor pool, we organized two New Developer Sprints and one Intermediate Sprint. Because we provided stipends for participants, admission was competitive and hence a non-trivial application process was needed. The goal of these one-week courses was to teach software development and open-source contribution skills to interested community members. Holding the courses online (and with flexible work times to accommodate different time zones) allowed for the participation of scientists from all over the world. In total, 29 participants (12 of them female) took part in one of the two New Developer Sprints, coming from eleven countries in Europe, North and South America, and South Asia. While the first sprint had a large cohort (23 participants), the second sprint was scaled back to only 7 participants, for financial and logistical reasons. Eleven participants of the two New Developer Sprints returned for the Intermediate Sprint in 2023.

We used a video conferencing platform that supports fast, high-resolution screen-sharing and the creation of an arbitrary number of voice/video/screen "rooms" for participants to work in. Each sprint had an associated kanban-style project board to facilitate finding and tracking the pieces of work to be done. Both types of sprint involved participants pair-programming with each other and with mentors. Sprint mentors (MNE-Python maintainers) were available in shifts, waiting in an "available mentors" room for participants to ask questions, request reviews or pair-programming sessions, or seek their next piece of work. Short plenary lectures and tutorials augmented the time spent coding.

3.1. New Developer Sprints

In 2021 and 2022 we held New Developer Sprints, aimed at equipping users of our software with the knowledge necessary to become contributors. Since most of our users are from the fields of neuroscience or cognitive science, and many do not have formal training in computer science or programming, tasks like setting up a development environment, using git and GitHub, writing documentation and tests, and navigating reviews and CI logs are all knowledge gaps that prospective contributors must bridge. For the New Developer Sprints, our application process included questions designed to guarantee that applicants had, at minimum, a functioning development environment and an up-to-date clone of their own fork of the upstream MNE-Python repository.[3] We provided support to help applicants get this part right, and proactively reached out before the deadline if their submissions indicated a wrong or incomplete setup. By doing this, we avoided spending any sprint time on environment setup, git configuration, or other preliminaries.

The content of the New Developer Sprints was also curated in advance. Dozens of issues were tagged as reserved for the sprint, including several nearly-identical small documentation tasks spread across the codebase. Participants were given brief instructions in pair programming practices (for review of benefits, see B. Hanks, S. Fitzgerald, R. McCauley, L. Murphy, and C. Zander [31]), assigned a partner, and each pair was given several small documentation issues to work through. The purpose of these small issues was to repeatedly

[2]Theoretically, another option would be engaging a competent developer and teaching them how to think like a neuroscientist, but anecdotally the consensus among leaders of other scientific software projects seems to be that it is much easier to train scientists to develop software than *vice-versa*.

[3]Example application questions: (1) change directory to the root of your repository clone, and paste the output of the command `git remote -v`. (2) start an IPython session and run the commands `import mne; mne.sys_info()` and paste the output here.

expose participants to the full contribution life cycle: fetch upstream arrow.r make branch arrow.r write code arrow.r build docs/run tests arrow.r commit changes arrow.r push arrow.r open PR arrow.r check CIs arrow.r respond to reviews. After a day or so spent on the small documentation issues, nearly all participants took on more complex tasks involving editing module code, writing tests, or updating tutorials.

Each day, a short lecture or tutorial was provided by senior community members within and outside of academia. Lectures focused on career trajectory, experiences in the MNE-Python community, and how the speakers benefitted from being contributors. Tutorials focused on knowledge relevant to the contributing workflow: writing and running tests, checking code coverage, reading tracebacks, and similar topics.

3.2. Intermediate Sprint

In 2023, we held an Intermediate Sprint, open to alumni of the New Developer Sprints who wanted to advance their contributing skills. Applications were again competitive, stipends were provided, and the sprint was again held online with staggered work times to accommodate participants' time zones. Participants chose larger contributing projects in advance, and could propose their own project (in their application) or choose from a range of pre-selected projects. The projects were more complex and took participants several days if not the whole week to complete. During the sprint, they joined with each other or with mentors for pair-programming sessions. Hands-on contributing was complemented by daily short lectures on pertinent topics such as debugging, building documentation, deprecations, understanding output of the CIs we use, and more. While the New Developer Sprints had focused mostly on contribution skills, the projects at the Intermediate Sprint relied on neuroscience knowledge as well, making the experience more akin to the work done by seasoned contributors. Examples of sprint projects include how to handle bad channels when working with multiple recordings, adding functionality to equalize observation numbers ("epochs") for time-frequency-resolved data, fixing a bug in our permutation statistics code, or annotating eyetracking data.

3.3. Maintainer Onboarding

After completion of the Intermediate Sprint we sought funding to support the onboarding of new MNE-Python maintainers. We received funding in 2025, and began training three alumni of our Intermediate Sprint and one exceptionally good GSoC contributor. The funding again allowed us to provide substantial stipends to the trainees. For the first 6 months, trainees met weekly (online) with an experienced maintainer to discuss topics such as optimization, configuring CIs, conducting code reviews, advanced testing concepts (fixtures, mocking, parametrization), environment management, packaging, dependencies, refactoring, release processes, and similar topics.

After 6 months, onboarding activities transitioned from prepared lessons to a combination of three activities: triage and PR review sessions, planning and execution of non-trivial documentation improvements, and phased improvements to our CI configuration. Plans for the second year of onboarding include deeper engagement with our user forum (including development of a semi-automated approach to triaging user support queries), assessment and adoption of triage management tooling, involving the trainee maintainers in long-term planning discussions (roadmap and funding), and integrating the lessons learned in year 1 into our in-process onboarding curriculum (see Section 6 below).

In addition to working directly with experienced MNE-Python maintainers, the trainee maintainers have also attended a variety of workshops and short courses, including training in code-of-conduct response and enforcement, community management, and project

management. Our funding for this onboarding project also supports two in-person sprints to integrate the trainee maintainers with the existing maintainer team.

3.4. Curriculum

Throughout the onboarding activities, we are collecting notes and creating materials with the aim of publishing an MNE-Python maintainer onboarding curriculum. This documentation is meant to streamline future onboarding efforts, but experience so far shows that it will also precipitate changes to our Contributing Guide. The domain-general parts of this onboarding curriculum will be extracted and published separately, for use by the wider Scientific Python Ecosystem. This will complement existing resources such as The Turing Way (focused on reproducibility and collaboration), PyOpenSci (focused on DIY packaging of research code), and the Scientific Python Library Development Guide (focused on best-practice recommendations for maintainers, but misses some topics where differences between projects make such general recommendations impractical).[4]

4. Discussion

MNE-Python has been struggling to attract and retain contributor and maintainer talent. This is due to a mixture of factors: a small pool of qualified people, the unfavorable-to-open-source academic incentive structure, and our past recruitment practices.

4.1. Onboarding Results

Figure 1 shows the retention of sprint attendees as contributors to MNE-Python. For both sprints, around half of the attendees continued to contribute beyond the first year. Taking both sprints together, ten attendees contributed for two years or longer. In 2026, five and four years after the sprints, four attendees are still regularly contributing. Our contributor statistics show, furthermore, that eight sprint attendees are currently ranking in the top 10% of all-time contributors to the project.[5] This includes some contributors who have been active for a smaller number of years but have been extraordinarily productive in that time.

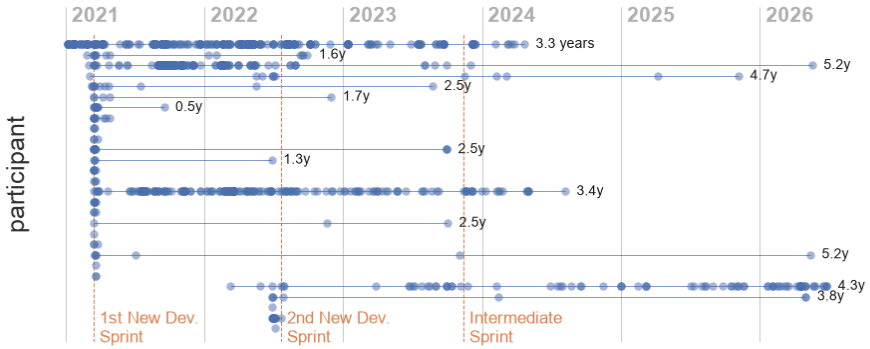


Figure 1. *Contributor retention after the sprints. Each row is a sprint attendee, and each dot is a contribution in one of seven MNE-Tools repositories (`mne-python`, `mne-bids`, `mne-connectivity`, `mne-gui-addons`, `mne-hfo`, `mne-icalabel`, `mne-qt-browser`). Vertical dashed lines show the dates of the two New Developer Sprints and the Intermediate Sprint. For participants who contributed at least 6 months after their sprint, the time span of their contributions is labeled.*

[4] For example, recommendations about triage practices strongly depend on the current size of a project's maintainer team, and the heterogeneity of the domain knowledge needed across different submodules of a package.

[5] Contribution statistics are quantified as lines added to or deleted from the repository, as in this script; results can be viewed at https://mne.tools/dev/credits/credit.html. We acknowledge that this approach provides an incomplete picture of contribution history, but is (at the time of writing) the best estimate we have available.

In contrast to our past network-based recruitment practices, the current approach has been bottom-up: first training users how to contribute, then upskilling contributors to facilitate repeat contributions, and finally providing intensive training in maintainer-specific skills. In our experience, providing education on how to contribute to open-source software, especially information specific to our project, greatly lowers the threshold for our users to be willing to attempt a contribution. This approach also allowed us to prioritize inclusivity in our recruitment and to address barriers that disproportionately impact underrepresented groups in our training. As of now, this has led to a slight increase in the diversity of our regular contributors and maintainers. Overall, we have retained roughly a third of the attendees for two years or longer. We do not have any insight into why long-term active attendees stop contributing, but assume that new careers, possibly outside of neuroscience, and the academic incentive system play a major role. However, that eight attendees are ranking among the top 10% contributors to MNE-Python and that four attendees are currently being onboarded to become maintainers exemplify the positive impact of our new onboarding approach on the project.

4.2. Onboarding Costs

The onboarding efforts, however, came with considerable monetary and personnel costs. The sprints and maintainer onboarding were funded by three separate grants over a six-year period, with a total cost exceeding $100,000. Beyond the monetary requirements (compensation for the trainees and key personnel), they were a huge investment of existing maintainers' time.[6] To spell it out: the time spent training new maintainers could not be used to fix bugs or make progress on our roadmap, thereby putting further strain on the maintenance of the software (at least in the short term). Nonetheless, looking ahead we still consider this a valuable investment that hopefully will make future maintenance easier by distributing it across a larger team, and by creating resources that make it easier to keep the size of that team stable going forward.

4.3. Onboarding Obstacles and Outlook

The incentive structure of academia still works against retaining our contributors and maintainers long-term, though the rise of academic OSPOs and recent efforts by organizations like ReSA, RDA, and CURIOSS seem to be gradually improving the situation. The funding landscape for scientific software also seems to be slowly improving, but in general still prioritizes new development over maintenance or technical debt reduction. We hope that by publicizing our onboarding curriculum and creating other "contributor ladder" resources, we will empower more users to self-educate about the open-source contribution process and the path to maintainership. This will hopefully increase the input stream of contributors to open-source scientific software projects, and may also increase retention: by making contribution easier through upskilling, hopefully each single contribution becomes less effortful and thus more likely to be attempted.

5. Funding acknowledgment

This project has been made possible by NSF POSE award 2449064, NIH R01NS104585, and by grant numbers [32] and 2021-237679 from the Chan Zuckerberg Initiative DAF, an advised fund of Silicon Valley Community Foundation (funder DOI 10.13039/100014989).

[6] Although we lack detailed records, we estimate 1200-1500 hours of maintainer time spent directly on organizing and carrying out the trainings, plus additional time spent on grantwriting, reporting, and related tasks.

References

[1] E. Larson *et al.*, "MNE-Python." [Online]. Available: https://doi.org/10.5281/zenodo.19666955

[2] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[3] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] L. Buitinck *et al.*, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.

[6] The Pandas Development Team, "pandas-dev/pandas: Pandas." [Online]. Available: https://doi.org/10.5281/zenodo.3509134

[7] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, Stéfan van der Walt and Jarrod Millman, Eds., 2010, pp. 56–61. doi: https://doi.org/10.25080/Majora-92bf1922-00a.

[8] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with python," in *9th Python in Science Conference*, 2010. doi: 10.25080/Majora-92bf1922-011.

[9] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: https://doi.org/10.1109/MCSE.2007.55.

[10] O. Moore, N. Jessurun, N. Nemitz, M. Chase, and L. Campagnola, "PyQtGraph - High Performance Visualization for All Platforms." [Online]. Available: https://doi.org/10.5281/zenodo.8133545

[11] B. Sullivan and A. Kaszynski, "PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK)," *Journal of Open Source Software*, vol. 4, no. 37, p. 1450, 2019, doi: 10.21105/joss.01450.

[12] A. Danial, "cloc: v2.10." [Online]. Available: https://doi.org/10.5281/zenodo.18364352

[13] A. Li *et al.*, "mne-connectivity." [Online]. Available: https://doi.org/10.5281/zenodo.19510920

[14] M. van Vliet, "MNE microstates: Microstate analysis for use with MNE-Python." [Online]. Available: https://github.com/wmvanvliet/mne_microstates

[15] V. Férat, M. Scheltienne, R. Kobler, A. Quinn, E. Larson, and L. Planchamp, "Pycrostates: a Python library to study EEG microstates." [Online]. Available: https://pycrostates.readthedocs.io/

[16] M. van Vliet *et al.*, "MNE-RSA: Representational Similarity Analysis on EEG, MEG and fMRI data." [Online]. Available: https://doi.org/10.5281/zenodo.17907540

[17] T. Dupré la Tour *et al.*, "pactools: estimate phase-amplitude coupling in neural time series." [Online]. Available: https://pactools.github.io/

[18] A. P. Rockhill *et al.*, "Intracranial Electrode Location and Analysis in MNE-Python," *Journal of Open Source Software*, vol. 7, no. 70, p. 3897, 2022, doi: 10.21105/joss.03897.

[19] R. Luke *et al.*, "Analysis methods for measuring passive auditory fNIRS responses generated by a block-design paradigm," *Neurophotonics*, vol. 8, no. 2, 2021, doi: 10.1117/1.NPh.8.2.025008.

[20] R. Höchenberger *et al.*, "OpenNeuro-Py." [Online]. Available: https://github.com/hoechenberger/openneuro-py

[21] D. A. Engemann *et al.*, "MNE-HCP." [Online]. Available: https://github.com/mne-tools/mne-hcp

[22] S. Appelhoff *et al.*, "MNE-BIDS: MNE-Python + BIDS = easy dataset interaction." [Online]. Available: https://doi.org/10.5281/zenodo.3891836

[23] K. J. Gorgolewski *et al.*, "The brain imaging data structure, a format for organizing and describing outputs of neuroimaging experiments," *Scientific Data*, vol. 3, no. 1, 2016, doi: 10.1038/sdata.2016.44.

[24] M. Scheltienne, E. Larson, A. Desvachez, and K. Lee, "MNE-LSL: Real-time framework integrated with MNE-Python for online neuroscience research through LSL-compatible devices," *Journal of Open Source Software*, vol. 10, no. 111, p. 8088, 2025, doi: 10.21105/joss.08088.

[25] R. Höchenberger *et al.*, "mne-tools/mne-bids-pipeline: v1.10.1." [Online]. Available: https://doi.org/10.5281/zenodo.19670260

[26] M. Jas, D. A. Engemann, Y. Bekhti, F. Raimondo, and A. Gramfort, "Autoreject: Automated artifact rejection for MEG and EEG data," *NeuroImage*, vol. 159, pp. 417–429, 2017, doi: https://doi.org/10.1016/j.neuroimage.2017.06.030.

[27] S. Appelhoff *et al.*, "PyPrep: A Python implementation of the Preprocessing Pipeline (PREP) for EEG data." [Online]. Available: https://pyprep.readthedocs.io/en/latest/

[28] G. Avelino, L. Passos, A. Hora, and M. T. Valente, "A novel approach for estimating Truck Factors," in *2016 IEEE 24th International Conference on Program Comprehension (ICPC)*, 2016, pp. 1–10. doi: 10.1109/ICPC.2016.7503718.

[29] B. U. Westner *et al.*, "Cycling on the Freeway: The perilous state of open-source neuroscience software," *Imaging Neuroscience*, vol. 3, p. imag_a_554, 2025, doi: 10.1162/imag_a_00554.

[30] D. Nafus, "'Patches don't have gender': What is not open in open source software," *New Media & Society*, vol. 14, no. 4, pp. 669–683, 2012, doi: 10.1177/1461444811422887.

[31] B. Hanks, S. Fitzgerald, R. McCauley, L. Murphy, and C. Zander, "Pair programming in education: a literature review," *Computer Science Education*, vol. 21, no. 2, pp. 135–173, 2011, doi: 10.1080/08993408.2011.579808.

[32] [Online]. Available: http://dx.doi.org/10.37921/890928yoegkg